

Rx Minom

Dokumentasi Teknis — Aplikasi Peningat & Pencatat Konsumsi Obat/Vitamin

Versi Final · Arsitektur REST API

Backend: Node.js, Express, SQLite (node:sqlite)
Frontend: HTML statis + JavaScript murni (fetch API)
Disusun untuk keperluan presentasi & pemahaman kode UAS

Daftar Isi

- 1. Gambaran Umum & Arsitektur (REST API)
- 2. Alur Request Umum
- 3. Struktur Folder Project
- 4. `db.js` — Skema Database
- 5. `server.js` — Pusat Kendali Aplikasi
- 6. `middleware/auth.js` — Proteksi API
- 7. `routes/api/auth.js` — Register, Login, Logout, Cek Sesi
- 8. `routes/api/medicines.js` — CRUD Obat/Vitamin
- 9. `routes/api/schedules.js` — Jadwal Konsumsi
- 10. Halaman HTML Statis (`public/*.html`)
- 11. JavaScript Sisi Browser (`public/js/*.js`) — Bedah Tiap File
- 12. Studi Kasus 1: Alur Lengkap Login
- 13. Studi Kasus 2: Alur Lengkap Tambah Obat & Tampil di Dashboard
- 14. Sistem Reminder & Countdown
- 15. Kapasitas & Batasan Sistem
- 16. Antisipasi Pertanyaan Dosen

1. Gambaran Umum & Arsitektur (REST API)

Minom versi final memakai pola **REST API + Static Frontend** — pola yang sama dipakai aplikasi web modern pada umumnya. Prinsip dasarnya: **server tidak lagi "menggambar" tampilan**, server hanya menyediakan data dalam format JSON. Yang menggambar tampilan adalah **JavaScript yang berjalan di browser**.

Lapisan	Teknologi	Tanggung Jawab
Frontend (statis)	HTML + CSS + JavaScript murni	Kerangka tampilan tetap, dikirim apa adanya oleh server
Frontend (dinamis)	JavaScript (<code>fetch API</code>)	Meminta data ke server, lalu membangun tampilan secara langsung di browser
Backend	Node.js + Express	Menyediakan endpoint API yang menerima request dan membalas JSON
Database	SQLite (<code>node:sqlite</code>)	Penyimpanan data permanen: user, obat, jadwal

Analogi sederhana: Bayangkan server sebagai "dapur restoran cepat saji" yang hanya menyerahkan makanan mentah lewat jendela (data JSON). Browser adalah "pelanggan" yang menerima bahan mentah itu dan menatanya sendiri di piring (menyusun tampilan HTML). Berbeda dengan restoran biasa (arsitektur lama) di mana dapur sudah menata semuanya di piring sebelum diserahkan.

Kenapa pola ini dipilih?

- **Pemisahan tanggung jawab lebih tegas:** server murni logic & data, frontend murni tampilan
- **Siap dikembangkan ke platform lain:** API yang sama bisa dipakai ulang untuk aplikasi mobile di masa depan, tanpa mengubah backend sama sekali
- **Interaksi terasa lebih cepat:** setelah kerangka halaman dimuat sekali, perpindahan data (misal hapus satu obat) tidak perlu memuat ulang seluruh halaman

2. Alur Request Umum

Karena tampilan dan data terpisah, ada **dua jenis request** yang terjadi di aplikasi ini:

Jenis 1: Meminta halaman (HTML/CSS/JS)

```
Browser minta /dashboard.html
|
v
Express (express.static) mengirim file HTML APA ADANYA
|
v
Browser merender kerangka HTML (masih KOSONG datanya)
|
v
Browser menjalankan <script> yang tertaut di HTML itu
```

Jenis 2: Meminta/mengirim data (API)

```
JavaScript di browser memanggil fetch('/api/...')
|
v
server.js menerima request, lewat middleware (baca JSON, cek session)
|
v
Diarahkan ke routes/api/ yang sesuai
|
v
Routes "ngobrol" ke db.js (SELECT/INSERT/UPDATE/DELETE)
|
v
Routes membalas dengan res.json({...})
|
v
JavaScript di browser menerima JSON, lalu membangun/mengubah
tampilan HTML secara langsung (document.createElement, innerHTML, dst)
```

Poin kunci untuk dipahami: Setiap halaman selalu memuat lewat Jenis 1 dulu (kerangka kosong), baru kemudian menjalankan satu atau lebih permintaan Jenis 2 untuk mengisi kontennya. Ini adalah pembeda utama dari arsitektur lama yang mengirim HTML sudah lengkap terisi dalam satu kali kirim.

3. Struktur Folder Project

```

minom/
|-- server.js           (nyalain server, daftar middleware & API routes)
|-- db.js              (blueprint & koneksi database)
|-- package.json
|
|-- middleware/
|   |-- auth.js         (requireLoginApi - proteksi endpoint API)
|
|-- routes/api/
|   |-- auth.js         (/api/register, /api/login, /api/logout, /api/me)
|   |-- medicines.js    (/api/medicines - GET, POST, DELETE)
|   |-- schedules.js    (/api/medicines/:id/schedules, /api/schedules/:id)
|
|-- public/             (SEMUA dikirim apa adanya oleh express.static)
|   |-- index.html      (pos jaga - cek login, lempar ke halaman sesuai)
|   |-- login.html + js/login.js
|   |-- register.html + js/register.js
|   |-- dashboard.html + js/dashboard.js
|   |-- riwayat.html + js/riwayat.js
|   |-- medicine-add.html + js/medicine-add.js
|   |-- medicine-schedule.html + js/medicine-schedule.js
|   |-- js/reminder.js   (countdown & notifikasi, jalan di semua halaman obat)
|   |-- js/bg-dots.js    (dekorasi visual, jalan di semua halaman)
|   |-- css/style.css
|
|-- database/
|   |-- minom.db         (file database asli, dibuat otomatis)

```

Perhatikan: tidak ada lagi folder `views/` berisi file `.ejs`. Semua tampilan sekarang murni file `.html` statis di dalam `public/`, dan kontennya "diisi belakangan" oleh file JavaScript pasangannya.

4. `db.js` — Skema Database

`db.js`

Bagian ini **tidak berubah** dari versi sebelumnya — database tetap dirancang dengan cara yang sama, karena pergantian arsitektur (EJS → REST API) hanya mengubah cara data *disajikan*, bukan cara data *disimpan*.

```

const { DatabaseSync } = require('node:sqlite');
const db = new DatabaseSync(dbPath);
db.exec('PRAGMA foreign_keys = ON');

```

Modul `node:sqlite` ini bawaan dari Node.js sendiri (sejak versi 22.5), sehingga tidak perlu instalasi library eksternal untuk urusan database.

Empat tabel yang didefinisikan

Tabel	Kolom Penting	Fungsi
<code>users</code>	id, nama, email (unik), password (hash)	Data akun
<code>medicines</code>	id, user_id, nama, dosis, jenis, tanggal_mulai, tanggal_selesai	Obat/vitamin milik satu user
<code>schedules</code>	id, medicine_id, jam_konsumsi	Jam-jam konsumsi untuk satu obat
<code>logs</code>	id, schedule_id, tanggal, status	Disiapkan strukturnya, belum dipakai fitur di frontend

Relasi berjenjang antar tabel:

```
users
  -- medicines (kolom user_id)
  -- schedules (kolom medicine_id)
```

```
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
```

ON DELETE CASCADE berarti kalau satu user dihapus, seluruh obat miliknya (dan jadwal di dalamnya) otomatis ikut terhapus — mencegah data "yatim piatu" yang tidak punya pemilik.

5. `server.js` — Pusat Kendali Aplikasi

`server.js`

```
const authApi = require('./routes/api/auth');
const medicinesApi = require('./routes/api/medicines');
const schedulesApi = require('./routes/api/schedules');

app.use(express.json()); // BARU: baca body JSON dari fetch()
app.use(express.urlencoded({ extended: true }));

app.use(session({ ... }));

app.use('/api', authApi);
app.use('/api', medicinesApi);
app.use('/api', schedulesApi);

app.use(express.static(path.join(__dirname, 'public')));
```

Baris	Penjelasan
<code>express.json()</code>	Middleware yang membaca body request berformat JSON. Ini WAJIB ada karena <code>fetch()</code> di JavaScript mengirim data sebagai JSON, berbeda dari form HTML biasa yang formatnya <code>urlencoded</code> .
<code>app.use('/api', ...)</code>	Semua request yang diawali <code>/api/</code> akan dicocokkan ke salah satu dari 3 file routing ini.
<code>app.use(express.static(...))</code>	Diletakkan setelah pendaftaran <code>/api</code> . Semua file di dalam <code>public/</code> otomatis bisa diakses lewat URL tanpa perlu route manual satu per satu.

Kenapa urutannya penting? Kalau `express.static` didaftarkan SEBELUM `/api`, request ke `/api/medicines` berisiko dicocokkan lebih dulu ke pencarian file bernama itu di folder `public/` — dan gagal, karena file seperti itu memang tidak ada di sana.

Tidak ada lagi route `/dashboard` di `server.js`

Berbeda dari versi sebelumnya yang punya route `app.get('/dashboard', ...)` lengkap dengan query database dan `res.render()`, di versi final ini **server.js tidak tahu apa-apa soal halaman dashboard**. Ia hanya mengirim file `dashboard.html` apa adanya lewat `express.static`. Semua logic pengambilan data obat sekarang murni tanggung jawab `routes/api/medicines.js` dan `public/js/dashboard.js`.

6. `middleware/auth.js` — Proteksi API

`middleware/auth.js`

```
function requireLoginApi(req, res, next) {
  if (req.session && req.session.userId) {
    return next();
  }
  return res.status(401).json({ error: 'Anda harus login dulu.' });
}

module.exports = { requireLoginApi };
```

Fungsi ini jauh lebih ringkas dari versi sebelumnya, tapi logikanya sama: cek apakah `req.session.userId` ada isinya.

Perbedaan penting dari versi lama: dulu kalau belum login, middleware langsung `res.redirect('/login')` — memaksa pindah halaman. Sekarang middleware ini hanya melindungi **endpoint API**, bukan halaman. Kalau belum login, ia membalas dengan **kode status HTTP 401** (Unauthorized) beserta pesan JSON. API tidak berhak memutuskan "pindah ke halaman mana" — itu murni keputusan JavaScript di sisi browser yang memanggilnya.

Kode `401` ini adalah bagian dari standar HTTP status code. Beberapa yang relevan di aplikasi ini:

Kode	Arti	Dipakai kapan di Minom
200	OK / berhasil	Semua respons sukses (login, tambah obat, dst)
400	Bad Request	Data yang dikirim user tidak valid (misal password kurang dari 6 karakter)
401	Unauthorized	Belum login, tapi mencoba akses endpoint yang butuh login
404	Not Found	Obat/jadwal yang dicari tidak ditemukan (atau bukan milik user tsb)
500	Server Error	Ada error tak terduga di server (jarang terjadi kalau validasi lengkap)

7. `routes/api/auth.js` — Register, Login, Logout, Cek Sesi

`routes/api/auth.js`

POST `/api/register`

```
router.post('/register', async (req, res) => {
  const { nama, email, password, konfirmasi_password } = req.body;

  if (!nama || !email || !password || !konfirmasi_password) {
    return res.status(400).json({ error: 'Semua field wajib diisi.' });
  }
  if (password.length < 6) {
    return res.status(400).json({ error: 'Password minimal 6 karakter.' });
  }
  if (password !== konfirmasi_password) {
    return res.status(400).json({ error: 'Konfirmasi password tidak cocok.' });
  }

  const existingUser = db.prepare('SELECT id FROM users WHERE email = ?').get(email);
  if (existingUser) {
    return res.status(400).json({ error: 'Email sudah terdaftar, silakan login.' });
  }

  const hashedPassword = await bcrypt.hash(password, 10);
  db.prepare('INSERT INTO users (nama, email, password) VALUES (?, ?, ?)')
    .run(nama, email, hashedPassword);

  return res.json({ success: true });
});
```

Validasi dilakukan berurutan — setiap `if` yang gagal langsung `return`, sehingga pengecekan berikutnya tidak perlu dijalankan (pola ini

disebut *early return*, membuat kode lebih mudah dibaca daripada bersarang dengan banyak `else`).

`bcrypt.hash(password, 10)` mengubah password asli menjadi teks hash satu arah. Angka `10` adalah *salt rounds* — tingkat kerumitan proses hashing. Password asli **tidak pernah disimpan** ke database, hanya hasil hash-nya.

POST /api/login

```
const user = db.prepare('SELECT * FROM users WHERE email = ?').get(email);
if (!user) {
  return res.status(400).json({ error: 'Email atau password salah.' });
}

const isValidPassword = await bcrypt.compare(password, user.password);
if (!isValidPassword) {
  return res.status(400).json({ error: 'Email atau password salah.' });
}

req.session.userId = user.id;
req.session.userName = user.nama;
return res.json({ success: true, nama: user.nama });
```

Detail keamanan yang bagus untuk disebut ke dosen: pesan error untuk "email tidak ditemukan" dan "password salah" sengaja dibuat **sama persis** ("Email atau password salah"). Ini mencegah orang lain menebak-nebak email mana saja yang terdaftar di sistem (kalau pesannya beda, orang bisa tahu "oh, email ini terdaftar tapi passwordnya yang salah").

`bcrypt.compare` mengecek kecocokan tanpa pernah "membalikkan" hash ke bentuk asli — yang dibandingkan adalah hasil hash dari password yang baru diketik, dengan hash yang tersimpan.

GET /api/me — endpoint paling sering dipanggil

```
router.get('/me', (req, res) => {
  if (req.session && req.session.userId) {
    return res.json({ loggedIn: true, nama: req.session.userName });
  }
  return res.json({ loggedIn: false });
});
```

Peran endpoint ini sangat penting: hampir setiap file JavaScript di `public/js/` memanggil `/api/me` di awal untuk dua tujuan sekaligus: (1) mengecek apakah user sudah login, dan (2) mengambil nama user untuk ditampilkan di header ("Halo, [nama]"). Endpoint inilah yang menggantikan peran proteksi otomatis yang dulu dilakukan middleware di level halaman.

POST /api/logout

```
router.post('/logout', (req, res) => {
  req.session.destroy(() => {
    res.json({ success: true });
  });
});
```

`req.session.destroy()` menghapus seluruh data session di server, sehingga cookie yang tersimpan di browser jadi tidak berguna lagi (tidak cocok dengan session manapun).

8. `routes/api/medicines.js` — CRUD Obat/Vitamin

`routes/api/medicines.js`

```
router.use(requireLoginApi);
```

Baris ini di paling atas file berarti: **semua endpoint di file ini** otomatis butuh login, tidak perlu ditulis satu-satu.

GET /api/medicines — ambil semua obat + jadwalnya sekaligus

```
router.get('/medicines', (req, res) => {
  const medicines = db.prepare(
    'SELECT * FROM medicines WHERE user_id = ? ORDER BY created_at DESC'
  ).all(req.session.userId);

  const result = medicines.map((med) => {
    const schedules = db.prepare(
      'SELECT * FROM schedules WHERE medicine_id = ? ORDER BY jam_konsumsi ASC'
    ).all(med.id);
    return { ...med, schedules };
  });

  res.json(result);
});
```

`.map()` membuat array baru di mana setiap obat "digabungkan" dengan daftar jadwalnya sendiri (`{ ...med, schedules }`) artinya: salin semua properti obat, lalu tambahkan properti baru `schedules`). Endpoint inilah yang dipanggil **baik** oleh `dashboard.js` maupun `riwayat.js` — keduanya menerima data mentah yang sama, lalu menyaringnya berbeda-beda di sisi browser.

POST /api/medicines — tambah obat baru

```
const info = db.prepare(
  `INSERT INTO medicines (user_id, nama, dosis, jenis, tanggal_mulai, tanggal_selesai, catatan)
  VALUES (?, ?, ?, ?, ?, ?, ?)`
).run(req.session.userId, nama, dosis || null, jenis, tanggal_mulai || null, tanggal_selesai || null, catatan || null);

res.json({ success: true, id: info.lastInsertRowid });
```

`req.session.userId` dimasukkan otomatis ke kolom `user_id` — user tidak pernah bisa "memilih" obat ini didaftarkan atas nama siapa. `info.lastInsertRowid` adalah ID baris yang baru saja dibuat, dikembalikan agar frontend tahu ID-nya kalau dibutuhkan.

DELETE /api/medicines/:id — hapus obat

```
router.delete('/medicines/:id', (req, res) => {
  db.prepare('DELETE FROM medicines WHERE id = ? AND user_id = ?')
    .run(req.params.id, req.session.userId);
  res.json({ success: true });
});
```

Poin keamanan penting: penghapusan memakai **dua syarat sekaligus** — ID obatnya cocok, *dan* obat itu benar-benar milik user yang login. Kalau user B mencoba menghapus obat milik user A (misalnya menebak-nebak angka ID di request), query ini tidak menemukan barisnya, sehingga tidak ada yang terhapus. `:id` pada path adalah *route parameter*, tertangkap sebagai `req.params.id`.

9. routes/api/schedules.js — Jadwal Konsumsi

routes/api/schedules.js

```
function getOwnedMedicine(medicineId, userId) {
  return db.prepare('SELECT * FROM medicines WHERE id = ? AND user_id = ?')
    .get(medicineId, userId);
}
```

Fungsi bantuan ini dipanggil di setiap endpoint jadwal untuk memastikan obat yang mau diberi jadwal benar-benar milik user yang login — pola keamanan yang sama seperti di `medicines.js`, dipakai ulang di sini.


```
router.post('/medicines/:id/schedules', (req, res) => {
  const medicine = getOwnedMedicine(req.params.id, req.session.userId);
  if (!medicine) return res.status(404).json({ error: 'Obat tidak ditemukan.' });

  const { jam_konsumsi } = req.body;
  if (!jam_konsumsi) {
    return res.status(400).json({ error: 'Jam wajib diisi.' });
  }

  const info = db.prepare('INSERT INTO schedules (medicine_id, jam_konsumsi) VALUES (?, ?)')
    .run(medicine.id, jam_konsumsi);

  res.json({ success: true, id: info.lastInsertRowid });
});
```

```
router.delete('/schedules/:id', (req, res) => {
  const schedule = db.prepare(
    `SELECT schedules.* FROM schedules
     JOIN medicines ON medicines.id = schedules.medicine_id
     WHERE schedules.id = ? AND medicines.user_id = ?`
  ).get(req.params.id, req.session.userId);

  if (!schedule) return res.status(404).json({ error: 'Jadwal tidak ditemukan.' });
  db.prepare('DELETE FROM schedules WHERE id = ?').run(schedule.id);
  res.json({ success: true });
});
```

Penghapusan jadwal memakai `JOIN` antara tabel `schedules` dan `medicines`, karena tabel `schedules` sendiri tidak menyimpan `user_id` secara langsung — kepemilikannya harus ditelusuri lewat obat induknya dulu.

10. Halaman HTML Statis (`public/*.html`)

Karakteristik utama file-file ini: **isinya sengaja "kosong" di bagian data**. HTML hanya berisi kerangka struktur, lalu diisi belakangan oleh JavaScript.

index.html — "Pos Jaga"

```
<script>
  fetch('/api/me')
    .then((res) => res.json())
    .then((data) => {
      window.location.href = data.loggedIn ? '/dashboard.html' : '/login.html';
    });
</script>
```

Halaman ini tidak menampilkan apa-apa ke user — begitu dibuka, langsung bertanya ke `/api/me`, lalu mengalihkan otomatis ke `dashboard.html` (kalau sudah login) atau `login.html` (kalau belum). Ini yang membuat mengetik alamat dasar aplikasi selalu "pintar" mengarahkan ke tempat yang benar.

dashboard.html — contoh kerangka kosong

```
<div id="medicine-list-aktif" class="medicine-list"></div>
<div id="empty-aktif" class="empty-state" style="display: none;">
  <p>Belum ada obat/vitamin yang aktif.</p>
</div>
```

Perhatikan `<div id="medicine-list-aktif"></div>` — kosong melompong di HTML. Inilah "wadah" yang nanti diisi oleh `dashboard.js` setelah data diterima dari API. Atribut `id` adalah "alamat" yang dipakai JavaScript untuk menemukan elemen ini (`document.getElementById('medicine-list-aktif')`).

Navigasi antar halaman

```
<nav class="main-nav">
  <a href="/dashboard.html" class="nav-btn active">Dashboard</a>
  <a href="/riwayat.html" class="nav-btn">Riwayat</a>
</nav>
```

Berbeda dari arsitektur lama di mana Riwayat adalah bagian dari halaman Dashboard yang sama, sekarang keduanya adalah **halaman HTML terpisah** yang saling ditautkan lewat navigasi biasa.

11. JavaScript Sisi Browser (`public/js/*.js`) — Bedah Tiap File

Perhatian umum: semua file di bagian ini *tidak berjalan di server*. Kode ini dikirim sebagai teks biasa oleh `express.static`, lalu dijalankan oleh browser setelah halaman HTML-nya dimuat.

login.js

```
form.addEventListener('submit', async function (event) {
  event.preventDefault(); // cegah browser reload halaman

  const data = { email: form.email.value, password: form.password.value };

  const response = await fetch('/api/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data),
  });

  const result = await response.json();

  if (!response.ok) {
    errorBox.textContent = result.error;
    errorBox.style.display = 'block';
    return;
  }

  window.location.href = '/dashboard.html';
});
```

Potongan	Penjelasan
<code>event.preventDefault()</code>	Mencegah perilaku default form HTML (yang biasanya me-reload halaman saat submit)
<code>JSON.stringify(data)</code>	Mengubah objek JavaScript menjadi teks JSON, format yang dipahami server
<code>await fetch(...)</code>	<code>await</code> membuat kode "menunggu" hasil dari server sebelum lanjut ke baris berikutnya, tanpa membekukan seluruh halaman
<code>response.ok</code>	Properti otomatis dari <code>fetch</code> : <code>true</code> kalau status HTTP di rentang 200-an (sukses), <code>false</code> kalau 400/401/500 dst

register.js

Pola strukturnya identik dengan `login.js` — ambil data form, `fetch('/api/register', ...)`, tangani error atau redirect ke `/login.html?registered=1` kalau sukses. Parameter `?registered=1` di URL dibaca oleh `login.js` lewat `URLSearchParams` untuk menampilkan pesan "Pendaftaran berhasil!".

dashboard.js — file paling kompleks

```
function buatKartuObat(med) {
  const card = document.createElement('div');
  card.className = 'medicine-card';
  card.setAttribute('data-times', med.schedules.map((s) => s.jam_konsumsi).join(','));
  card.setAttribute('data-nama', med.nama);

  card.innerHTML = `
    <span class="badge badge-${med.jenis}">${med.jenis}</span>
    <h3>${med.nama}</h3>
    ...
  `;
  return card;
}
```

Fungsi ini adalah pengganti langsung dari `<% medicines.forEach(...) %>` di versi EJS lama. Bedanya mendasar: dulu perulangan ini berjalan **di server** sebelum HTML dikirim; sekarang berjalan **di browser** setelah data JSON diterima. `document.createElement` dan `innerHTML` adalah cara JavaScript membangun elemen HTML secara manual, potong per potong, lalu ditempelkan ke halaman.

Atribut `data-times` dan `data-nama` yang disisipkan ke setiap kartu bukan untuk tampilan — ini adalah "titipan data" yang nantinya dibaca oleh `reminder.js` untuk menghitung countdown, tanpa `reminder.js` perlu memanggil API lagi secara terpisah.

```
async function muatDataObat() {
  const response = await fetch('/api/medicines');
  const medicines = await response.json();

  const hariIni = new Date().toISOString().slice(0, 10);
  const aktif = medicines.filter((m) => !m.tanggal_selesai || m.tanggal_selesai >= hariIni);

  listAktifEl.innerHTML = '';
  aktif.forEach((med) => listAktifEl.appendChild(buatKartuObat(med)));
}
```

Logic pemisahan "obat aktif vs obat selesai" yang dulu dilakukan di `server.js` (pakai `.filter()` di JavaScript sisi server), sekarang **dipindah ke browser**. Server hanya mengembalikan semua obat mentah-mentah; halaman inilah yang memutuskan mana yang ditampilkan.

```
async function init() {
  const response = await fetch('/api/me');
  const data = await response.json();

  if (!data.loggedIn) {
    window.location.href = '/login.html';
    return;
  }

  userNamaEl.textContent = data.nama;
  muatDataObat();
}

init();
```

Baris `init()` di paling akhir file adalah yang **menjalankan semua ini** begitu skrip dimuat. Pola "cek login dulu, baru muat data" ini berulang di hampir semua file JS halaman — inilah pengganti proteksi otomatis yang dulu dilakukan middleware server.

riwayat.js

Strukturnya nyaris identik dengan `dashboard.js`, hanya beda di filter:

```
// dashboard.js:
const aktif = medicines.filter((m) => !m.tanggal_selesai || m.tanggal_selesai >= hariIni);
// riwayat.js:
const riwayat = medicines.filter((m) => m.tanggal_selesai && m.tanggal_selesai < hariIni);
```

Keduanya memanggil endpoint API yang **persis sama** (`/api/medicines`), lalu menyaring hasilnya secara berbeda di sisi browser masing-

masing.

medicine-add.js

Mengumpulkan data form, `fetch('/api/medicines', { method: 'POST', ... })`, lalu redirect ke `/dashboard.html` kalau berhasil. Polanya sama dengan `login.js` / `register.js`.

medicine-schedule.js

```
const params = new URLSearchParams(window.location.search);
const medicineId = params.get('id');

if (!medicineId) {
  window.location.href = '/dashboard.html';
}
```

Halaman ini butuh tahu "obat mana yang sedang diatur jadwalnya" — informasi ini dikirim lewat parameter URL, misalnya `medicine-schedule.html?id=3`. `URLSearchParams` adalah cara standar JavaScript membaca bagian `?key=value` dari alamat browser. Kalau parameter `id` tidak ada sama sekali, halaman otomatis melempar balik ke dashboard (mencegah halaman kosong tanpa konteks).

```
async function muatData() {
  const response = await fetch(`/api/medicines/${medicineId}`);
  if (!response.ok) {
    window.location.href = '/dashboard.html';
    return;
  }
  const medicine = await response.json();
  ...
}
```

Kalau API membalas dengan status bukan 200 (misalnya 404 karena obat tidak ditemukan atau bukan milik user ini), halaman langsung dialihkan balik — mencegah user "nyasar" mengatur jadwal obat yang sebenarnya tidak bisa diakses.

bg-dots.js — dekorasi visual

```
const JUMLAH_DOT = 25;
for (let i = 0; i < JUMLAH_DOT; i++) {
  const dot = document.createElement('span');
  dot.style.left = `${Math.random() * 100}%`;
  dot.style.animationDuration = `${10 + Math.random() * 15}s`;
  ...
  container.appendChild(dot);
}
```

Murni dekorasi — membuat 25 titik kecil yang "mengambang" naik di latar belakang setiap halaman, posisi dan kecepatan masing-masing diacak lewat `Math.random()`. Tidak ada logic fungsional di sini, hanya nilai tambah visual yang dijalankan di semua halaman untuk konsistensi tampilan.

12. Studi Kasus 1: Alur Lengkap Login

1. User buka /login.html
-> express.static kirim file HTML apa adanya (form kosong)
2. Browser jalankan <script src="/js/login.js">
3. User isi email & password, klik "Masuk"
4. login.js: event.preventDefault() (cegah reload halaman)
5. login.js: fetch('/api/login', { method: 'POST', body: JSON... })
6. server.js: request masuk, express.json() baca body-nya
7. server.js: diarahkan ke routes/api/auth.js
8. auth.js: cari user berdasar email -> db.prepare(SELECT...).get(email)
9. auth.js: bcrypt.compare(password, user.password)
10. auth.js: COCOK -> req.session.userId = user.id
11. auth.js: res.json({ success: true, nama: user.nama })
12. login.js: response.ok bernilai true
13. login.js: window.location.href = '/dashboard.html'
14. Browser pindah ke dashboard.html, siklus halaman baru dimulai

13. Studi Kasus 2: Alur Lengkap Tambah Obat & Tampil di Dashboard

1. User di /medicine-add.html, isi form, klik "Simpan"
2. medicine-add.js: event.preventDefault(), kumpulkan data form
3. medicine-add.js: fetch('/api/medicines', { method: 'POST', body: JSON... })
4. server.js: request masuk -> express.json() -> requireLoginApi cek session
5. routes/api/medicines.js: db.prepare(INSERT...).run(req.session.userId, ...)
-> data masuk ke minom.db
6. routes/api/medicines.js: res.json({ success: true, id: ... })
7. medicine-add.js: window.location.href = '/dashboard.html'
8. Browser minta /dashboard.html -> express.static kirim HTML kosong
9. dashboard.js: init() jalan -> fetch('/api/me') -> loggedIn true
10. dashboard.js: muatDataObat() -> fetch('/api/medicines')
11. routes/api/medicines.js: SELECT semua obat milik user ini (termasuk yang baru)
+ JOIN jadwal masing-masing lewat .map()
12. dashboard.js: terima array JSON, filter yang "aktif"
13. dashboard.js: forEach -> buatKartuObat() -> appendChild ke halaman
14. reminder.js: baca atribut data-times di kartu baru, mulai hitung countdown

Perbandingan dengan versi EJS lama: pada versi lama, langkah 8-13 di atas terjadi *di server* dalam satu kali proses sebelum HTML dikirim (`res.render()`). Pada versi final ini, HTML dikirim kosong dulu (langkah 8), baru diisi belakangan lewat panggilan API terpisah (langkah 9-13) — dua tahap yang jelas terpisah.

14. Sistem Reminder & Countdown

public/js/reminder.js

File ini **tidak berubah secara fundamental** dari versi sebelumnya, hanya mendapat tambahan fitur highlight "due soon".

```
setInterval(cekSemuaJadwal, 1000);
```

Menjalankan fungsi `cekSemuaJadwal` setiap 1 detik, selama halaman dashboard/riwayat masih terbuka.

```
function cariJadwalBerikutnya(times, now) {
  times.forEach((jamStr) => {
    const [h, m] = jamStr.split(':').map(Number);
    const waktuJadwal = new Date(now);
    waktuJadwal.setHours(h, m, 0, 0);
    if (waktuJadwal <= now) {
      waktuJadwal.setDate(waktuJadwal.getDate() + 1);
    }
    ...
  });
}
```

Menghitung jadwal terdekat berikutnya. Kalau jadwalnya jam 08:00 tapi sekarang sudah lewat jam itu, otomatis dianggap jadwal berikutnya adalah besok jam 08:00.

```
const BATAS_DUE_SOON_MS = 30 * 60 * 1000; // 30 menit
...
card.classList.toggle('due-soon', sisaMs <= BATAS_DUE_SOON_MS);
```

Fitur baru: kartu obat mendapat class CSS `due-soon` secara otomatis kalau jadwal berikutnya kurang dari 30 menit lagi, memberi highlight visual di CSS supaya user langsung tahu obat mana yang hampir waktunya.

Batasan penting untuk didemokan dengan jujur: reminder ini berjalan **selama tab browser terbuka**. Ini bukan notifikasi push sungguhan seperti WhatsApp yang tetap masuk walau aplikasi ditutup total — itu membutuhkan infrastruktur tambahan (service worker, VAPID key, background scheduler) yang berada di luar cakupan tugas ini.

15. Kapasitas & Batasan Sistem

Aspek	Kondisi Saat Ini	Implikasi
Penyimpanan data	SQLite, satu file	Mampu menampung jutaan baris, jauh melebihi kebutuhan demo/tugas
Session login	Disimpan di memori server (<code>MemoryStore</code>)	Kalau server di-restart, semua user otomatis "logout". Tidak masalah untuk demo; produksi sungguhan biasanya memakai store yang persisten
Query database	<code>node:sqlite</code> bersifat <i>synchronous/blocking</i>	Request diproses berurutan, tidak masalah untuk beberapa user, kurang ideal untuk trafik sangat tinggi bersamaan
Komunikasi data	REST API berbasis JSON	Bisa dipakai ulang oleh platform lain (misal aplikasi mobile) tanpa mengubah backend

Kesimpulan untuk dijawab ke dosen: "Untuk skala tugas dan demo, sistem ini tidak akan pernah mencapai batasnya. Justru dengan pola REST API, aplikasi ini lebih siap dikembangkan lebih lanjut dibanding pendekatan render-di-server, karena data dan tampilan sudah terpisah sejak awal."

16. Antisipasi Pertanyaan Dosen

Kemungkinan Pertanyaan	Jawaban Singkat
Kenapa pakai pola REST API, bukan render langsung di server?	Pemisahan yang lebih tegas antara data dan tampilan, serta API yang sama bisa dipakai ulang untuk platform lain (misal mobile) di masa depan tanpa mengubah backend.
Apa itu REST API?	Pola komunikasi di mana server menyediakan data lewat alamat (endpoint) tertentu, dibalas dalam format JSON, dan client (browser/aplikasi) yang menentukan bagaimana data itu ditampilkan.
Kenapa ada <code>fetch()</code> di mana-mana?	<code>fetch()</code> adalah fungsi bawaan JavaScript untuk mengirim request HTTP ke server tanpa perlu me-reload halaman — inti dari interaksi dinamis di frontend.
Bagaimana keamanan password?	Di-hash memakai bcrypt, algoritma satu arah, sehingga tidak bisa dikembalikan ke teks asli walau database

	bocor.
Bagaimana proteksi halaman kalau tidak ada middleware server?	Setiap file JavaScript halaman memanggil <code>/api/me</code> di awal untuk memverifikasi status login; kalau belum login, JavaScript sendiri yang mengalihkan ke halaman login.
Apakah reminder tetap jalan kalau aplikasi ditutup?	Untuk saat ini, reminder berjalan selama tab browser terbuka. Notifikasi push sungguhan butuh infrastruktur tambahan (service worker, VAPID key) di luar cakupan tugas ini.
Berapa banyak user yang bisa ditampung?	Secara penyimpanan data, praktis tidak terbatas untuk skala ini. Keterbatasan baru muncul di sisi session & trafik bersamaan dalam skala besar, di luar kebutuhan tugas ini.
Kenapa namanya Minom?	Dari kata "minum" — karena inti aplikasi ini soal ketepatan waktu minum obat dan vitamin.

Saran praktis: latihan demo minimal satu kali sebelum hari-H, khususnya bagian reminder — sesuaikan jam yang diinput di form jadwal dengan waktu demo (1–2 menit ke depan) agar notifikasi benar-benar muncul. Buka juga tab "Network" di DevTools browser (F12) sebagai bahan tambahan kalau dosen bertanya soal cara kerja API secara langsung — kamu bisa tunjukkan request/response JSON yang benar-benar terjadi saat berinteraksi dengan aplikasi.